

Evaluating Matrix Tiling Multiplication For Asymmetric Data Cost

Project Aleph Spring 2026

Regev Ginzburg, Areg Mikhtyrian

September 29, 2026

Contents

1	Introduction	1
1.1	Background: Caches, Memory, Matrix Multiplication	1
1.2	Theoretical Results	1
1.3	Importance Of Asymmetric Data Access Model	1
2	Methodology	2
2.1	<code>asymm_tiling</code> Simulator	2
2.1.1	Our Model	2
2.1.2	Architecture	3
2.1.3	Correctness	4
2.2	Simulation Parameters: Soundness And Real-Life References	5
3	Experiments	6
3.1	Reproducing The Original Paper	6
4	Findings And Discussion	7
5	Conclusion	8
6	Appendix	9
6.1	Implementation & Build Environment	9

1 Introduction

1.1 Background: Caches, Memory, Matrix Multiplication

1.2 Theoretical Results

1.3 Importance Of Asymmetric Data Access Model

Today, in light of development of algorithms for randomized numerical algebra (for example, the paper [Randomized Numerical Linear Algebra \(2023\)](#)) there is a case for generating low-precision matrices instead of traditional access from memory. Not only generated matrix is allowed to be of lower precision (which has been shown theoretically and by our experiments in Section 3.1 to reduce cache traffic

2 Methodology

2.1 `asymm_tiling` Simulator

To evaluate matrix multiplication under asymmetric data access costs, we designed and implemented `asymm_tiling`, a modular, cycle-accurate architectural simulator written in C++20. The simulator models the execution of General Matrix Multiply (GEMM) operations across configurable multi-level cache hierarchies, comparing conventional DRAM-backed dataflows against hardware-accelerated on-the-fly streaming.

2.1.1 *Our Model*

We model mixed-precision GEMM computing $C = A \cdot B$, where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}$$

In our asymmetric setup, input matrix A and accumulator/output matrix C are represented in high precision (e.g., FP32, INT32 or even INT64) we define their precision to be P_A . Matrix B on the other hand, resides in low precision (e.g., INT8) and we define its precision to be P_B . We define the precision asymmetry ratio as:

$$\rho = \frac{P_B}{P_A} \leq 1$$

We model two distinct operational paradigms for matrix element access:

1. **Conventional DRAM-backed Access:** Both operands A and B reside in simulated main memory and must be fetched through the memory hierarchy (L_1 , L_2 , L_3 ,... and DRAM). Under this regime, reducing P_B reduces the cache footprint of the matrices and their bandwidth demand by a factor of $\frac{1}{\rho}$.
2. **Hardware PRNG-FIFO Streaming:** Matrix B is generated on-the-fly in dedicated hardware from compact seeds that are stored in memory. It's generated via a Pseudo-Random Number Generator (PRNG) coupled to an asynchronous FIFO queue. Under this model, the elements of matrix B bypass the cache and DRAM subsystem entirely, incurring zero cache pollution and near-zero memory traffic.

The overall execution time T normalized per Multiply-Accumulate operation (MAC) is governed by two concurrent processes: memory latency and hardware generation rate:

$$\frac{T}{MNK} = \max\left(\alpha(T_M, T_N), \frac{g_c}{T_M}\right)$$

where:

- $\alpha(T_M, T_N)$ is the **memory-bound** cost (cycles per MAC) determined by cache miss rates and DRAM transfer latencies for a given tile shape ($T_M \times T_N$).
- g_c is the **hardware PRNG generation latency** (cycles per element of B). Because each element of B is reused T_M times across the rows of A in weight-stationary dataflow, the generation cost amortizes to $\frac{g_c}{T_M}$ cycles per MAC.

- Finally, the max operator reflects the asynchronous overlap between background FIFO generation and foreground compute/memory access.

2.1.2 Architecture

The simulator is structured into decoupled, modular subsystems synchronized by a centralized event clock:

- **MultiUnit (Compute & Tiling Engine):** Orchestrates the 3D loop nest (M, K, N) partitioned into macro-tiles (T_M, T_K, T_N) and micro register-tiles $(R_M \times R_N, \text{typically } 4 \times 4)$. It supports both **Output-Stationary** (C -stationary, accumulating outputs locally while streaming inputs) and **Weight-Stationary** (B -stationary, holding stationary weight tiles to maximize B reuse) dataflows.
- **PrngFifo (PRNG-FIFO Hardware Accelerator):** Models an asynchronous generator unit with finite FIFO queue capacity. When the compute core requests an element via `pop()`, the unit checks queue availability; if the FIFO is starved, the simulator accumulates cycle-accurate pipeline stall penalties ($\max(0, g_c - \Delta t)$) and advances the global clock.
- **CacheUnit (Multi-Level Cache Controller):** Coordinates an extensible Chain-of-Responsibility hierarchy of arbitrary depth $(L_1, L_2, \dots, L_k, \text{DRAM})$:
 - **CacheLevel:** Manages tag/index/offset address translation, hit latency accounting, and recursive backfill on misses.
 - **Set and ReplacementPolicy:** Manages associative way storage backed by pluggable eviction policies. By default, an efficient contiguous-vector LRU policy (`LruPolicy`) tracks line access orders with zero pointer overhead.
 - **Write Policies:** Implements configurable Write-Allocate (fetch-on-write with dirty line writeback) and No-Write-Allocate (write-around) policies.
- **Clock (Cycle Synchronization):** A centralized simulation clock (`Clock`) that advances monotonically with MAC compute latencies, cache lookup costs, DRAM transfer penalties, and PRNG-FIFO pipeline stall cycles.
- **Registry (Statistics Collection):** A centralized metrics registry (`Registry`) that records and computes dynamic simulation statistics (e.g., hits, misses, local/global miss rates, FIFO stall rates, total cycles, and average access times) via registered callbacks, decoupling statistics gathering from core simulation logic.
- **Config and MyOptions (Configuration & Parameter Management):** Implements a two-tiered parameterization system. `Config` loads baseline architectural profiles from declarative TOML files (`config.toml`), while `MyOptions` (extending `CLI:App`) parses command-line arguments to dynamically override parameters on the fly, enabling scripted parametric sweeps without recompiling the simulator.

The relationship between these subsystems is illustrated in the UML class diagram in Figure 1.

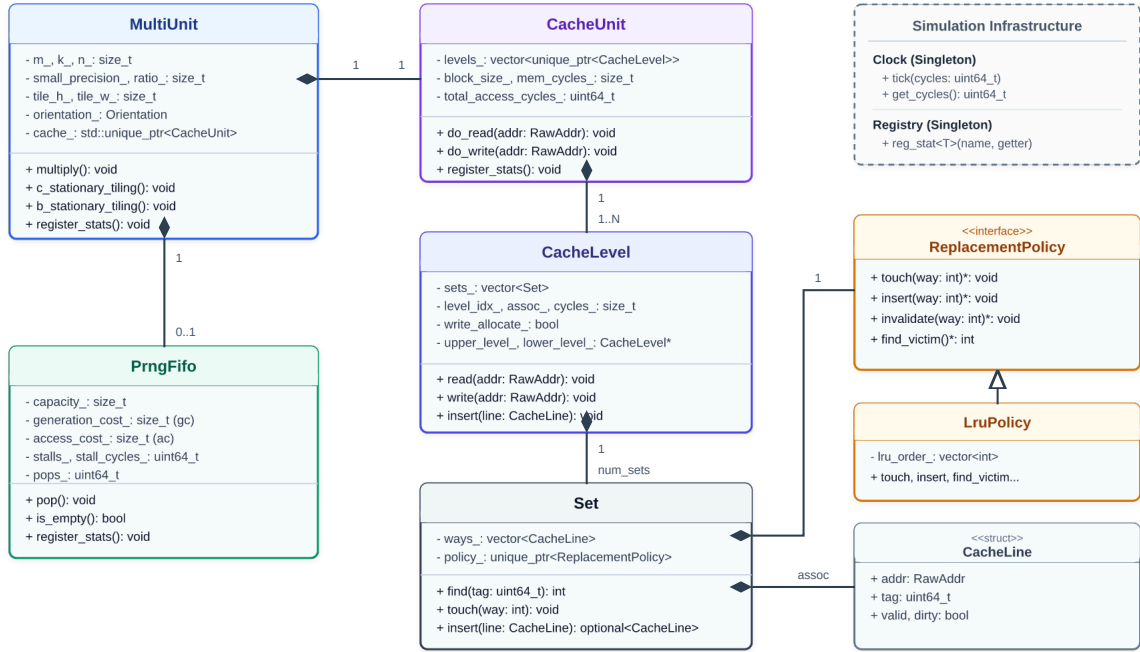


Figure 1: UML class diagram of the *asymm_tiling* simulator, illustrating the separation of the computation engine (*MultiUnit*), accelerator pipeline (*PrngFifo*), modular cache hierarchy (*CacheUnit*), and eviction policies.

To ensure high performance, maintainability, and clean separation of concerns, the simulator is implemented in standard C++20 and utilizes lightweight, header-only third-party libraries for infrastructure: `CLI11` for parsing command-line parameters, `toml++` for configuration file management, and `fmt/spdlog` for fast formatted console and log output. Detailed build instructions, compiler requirements, and library dependencies are summarized in Section 6.1.

2.1.3 Correctness

Simulator fidelity is validated through three complementary verification layers:

1. **Memory Access Invariants:** We enforce strict conservation laws across the cache hierarchy for every simulation run:

$$\text{Accesses}(L_k) = \text{Hits}(L_k) + \text{Misses}(L_k)$$

$$\text{Accesses}(L_{k+1}) = \text{Misses}(L_k)$$

$$\text{Global Miss Rate}(L_k) = \prod_{i=1}^k \text{Local Miss Rate}(L_i)$$

2. **FIFO State Consistency:** The accelerator validates queue invariants across pops, verifying that the total number of consumed elements matches the inner loop count ($MN \frac{K}{T_M}$ or KN), and that stall cycles align exactly with generation deficit periods ($g_c > a$).
3. **Theoretical Limit Validation:** In boundary conditions (e.g., $g_c = 0$, infinite cache capacity, or single-cycle memories), the simulator matches closed-form analytical predictions and reproduces the theoretical transition cliffs observed when matrix tiles exceed L_1 cache bounds.

2.2 Simulation Parameters: Soundness And Real-Life References

3 Experiments

3.1 Reproducing The Original Paper

4 Findings And Discussion

5 Conclusion

6 Appendix

6.1 Implementation & Build Environment

The full simulator source code, build configuration, and experiment reproduction scripts are available on GitHub:

https://github.com/gevorgyg/asymm_tiling

The simulator is built with CMake (version ≥ 3.16) targeting the C++20 language standard. The implementation avoids heavy runtime frameworks and utilizes header-only libraries:

- **CLI11**: Provides command-line option parsing with automatic type conversion and validation.
- **toml++**: Parses TOML configuration files (`config.toml`), allowing experiment parameters to be specified declaratively without recompilation.
- **fmt & spdlog**: High-performance structured logging library used to log cycle events and format aligned metrics tables.